

Worksheet 1

Aim:

To design, develop, and deploy **FreshMart**—a responsive, interactive, and premium **Online Grocery Delivery System** using a web frontend (HTML5, CSS3, Bootstrap 5, JavaScript) integrated with a local database backend (Node.js, Express, SQLite3) to enable real-time product filtering, quick-view details display, shopping cart management, promo code calculation, local persistence, and transaction checkout tracking.

Procedure:

Phase 1: Frontend & UI Design

1. Define visual design variables (colors, fonts, shadow tokens, glassmorphism filters) in `css/styles.css`.
2. Layout the structure of the application inside `index.html` using Bootstrap 5's responsive grid system, implementing a top announcement bar, glassmorphism navbar, promotional offer banners, category filter tabs, and a footer.
3. Integrate an Offcanvas shopping cart drawer, a Quick View product modal, and Toast notifications container for native visual feedback.

Phase 2: Client-side Interactivity (JavaScript)

1. Initialize filter states (category, search queries, brand selections, price range constraints, and sorting preferences).
2. Render product cards dynamically based on active filters and handle UI feedback (added animation tags, scale transitions on the cart badge).
3. Bind event handlers to the product card Quick View anchors to dynamically populate and launch the details modal with a quantity selector.
4. Manage shopping cart states inside local memory and synchronize with the browser's `localStorage` to preserve items across reloads.

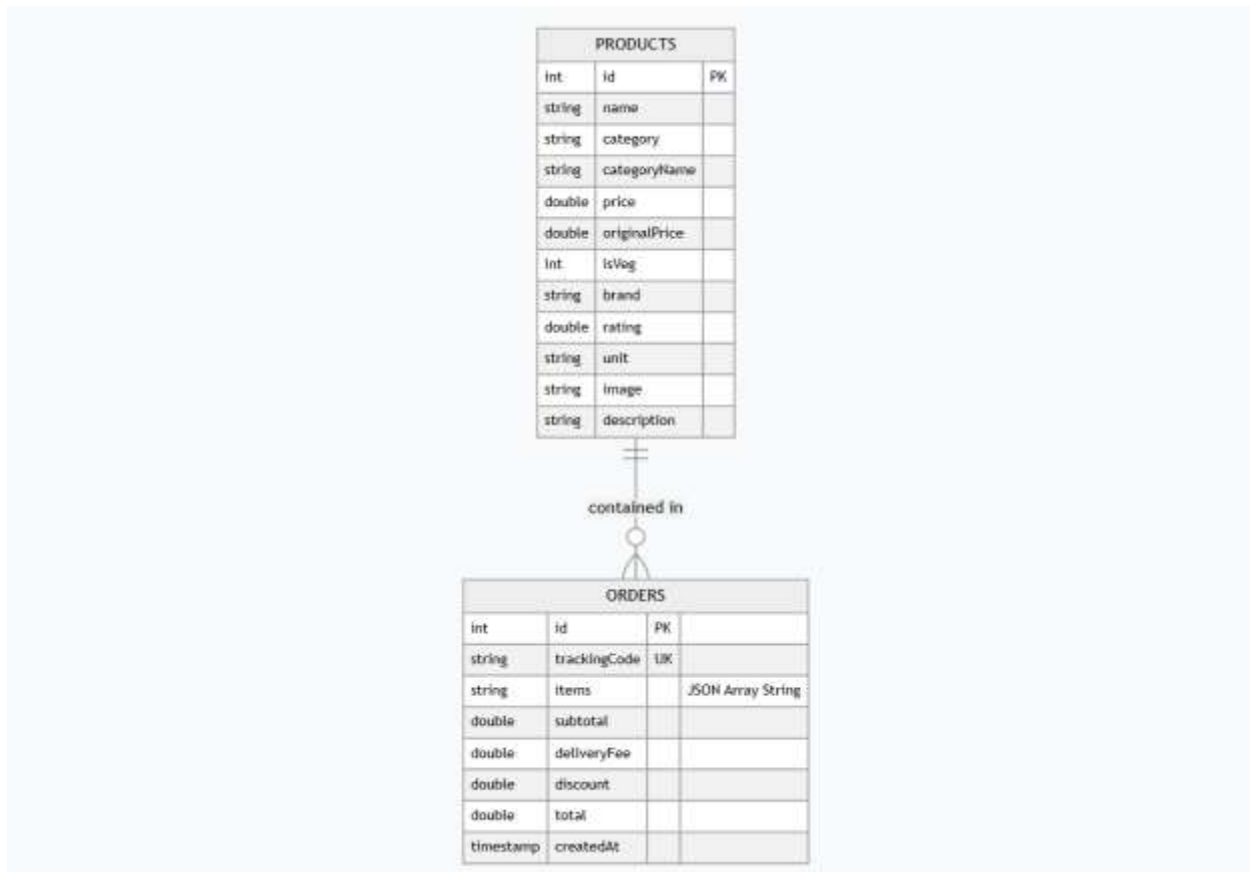
Phase 3: Server & Database Integration

1. Set up a Node.js runtime inside the `/server` directory and declare dependencies (`express`, `cors`, `sqlite3`) in `server/package.json`.
2. Write `server/db.js` to configure SQLite, establish connection schemas for products and orders tables, and implement auto-seeding behavior to insert raw catalog data on first boot.
3. Develop REST API routes inside `server/server.js`:
 - `GET /api/products`: queries and maps catalog table data.
 - `POST /api/orders`: validates purchase data, inserts transactional records, and responds with unique tracking IDs.

- Update frontend fetch operations inside js/app.js to pull products from the server and POST cart details upon order checkout.

ER and DFD Diagram:

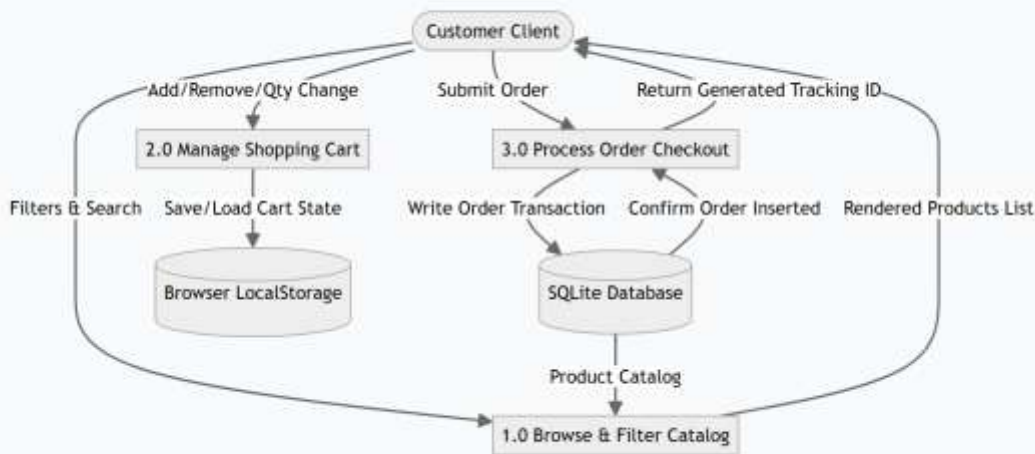
1. Entity-Relationship (ER) Diagram



2. Data Flow Diagram (DFD Level 0 - Context Diagram)



3. Data Flow Diagram (DFD Level 1 - Detailed Diagram)



Coding Sample (For All Modules):

• Front End (JavaScript)

Key logic segments from js/app.js:

1. Fetching Catalog Dynamically

```

async function fetchProducts() {
  try {
    const response = await fetch('http://localhost:5000/api/products');
    if (!response.ok) throw new Error('API response error');
    const data = await response.json();

    if (Array.isArray(PRODUCTS_DATA)) {
      PRODUCTS_DATA.length = 0;
      PRODUCTS_DATA.push(...data);
    }
  } catch (err) {
    console.warn('Failed to fetch products from backend, falling back to static local data:', err);
  }
}
  
```

```
}  
}
```

2. Cart Drawer State & Total Recalculation

```
function updateCartUI() {  
  const totalItems = cart.reduce((sum, item) => sum + item.quantity, 0);  
  if (cartCountEl) cartCountEl.textContent = totalItems;  
  
  const cartItemsList = document.getElementById('cartItemsList');  
  if (!cartItemsList) return;  
  
  if (cart.length === 0) {  
    cartItemsList.innerHTML = `

Your cart is empty.</p>`;  
    updateTotals(0);  
    return;  
  }  
  
  cartItemsList.innerHTML = "";  
  cart.forEach(item => {  
    const product = PRODUCTS_DATA.find(p => p.id === item.id);  
    if (!product) return;  
    // Append row layout with quantity increment/decrement controls  
  });  
  
  const subtotal = cart.reduce((sum, item) => {  
    const product = PRODUCTS_DATA.find(p => p.id === item.id);  
    return sum + (product ? product.price * item.quantity : 0);  
  }, 0);  
  updateTotals(subtotal);  
}


```

3. Client POST Order Checkout

```
const response = await fetch('http://localhost:5000/api/orders', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    items: cart,
    subtotal,
    deliveryFee,
    discount,
    total
  })
});
const orderData = await response.json();
// Populate success tracker modal using orderData.trackingCode
```

• Back End (Node.js & SQLite)

Key server configuration files:

1. SQLite Schema Configuration & Seeding (server/db.js)

```
const sqlite3 = require('sqlite3').verbose();
const db = new sqlite3.Database(path.join(__dirname, 'database.sqlite'));

db.serialize(() => {
  db.run(`
    CREATE TABLE IF NOT EXISTS products (
      id INTEGER PRIMARY KEY,
      name TEXT NOT NULL,
      price REAL NOT NULL,
      originalPrice REAL,
      isVeg INTEGER NOT NULL,
```

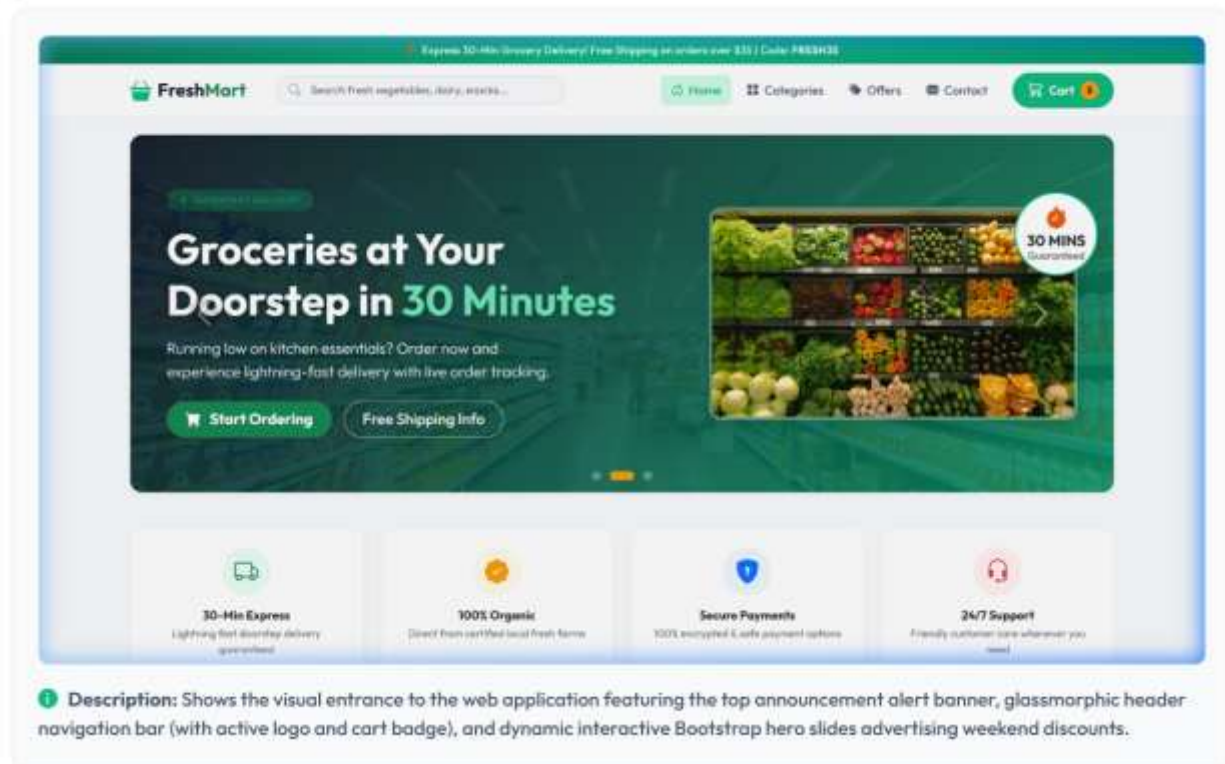
```
brand TEXT,  
image TEXT  
)  
`);  
  
db.get('SELECT COUNT(*) AS count FROM products', (err, row) => {  
  if (row.count === 0) {  
    const stmt = db.prepare(`INSERT INTO products (id, name, price, originalPrice, isVeg,  
brand, image) VALUES (?, ?, ?, ?, ?, ?, ?)`);  
    PRODUCTS_DATA.forEach(p => stmt.run(p.id, p.name, p.price, p.originalPrice, p.isVeg ? 1  
: 0, p.brand, p.image));  
    stmt.finalize();  
  }  
});  
});
```

2. Express Server Routes (server/server.js)

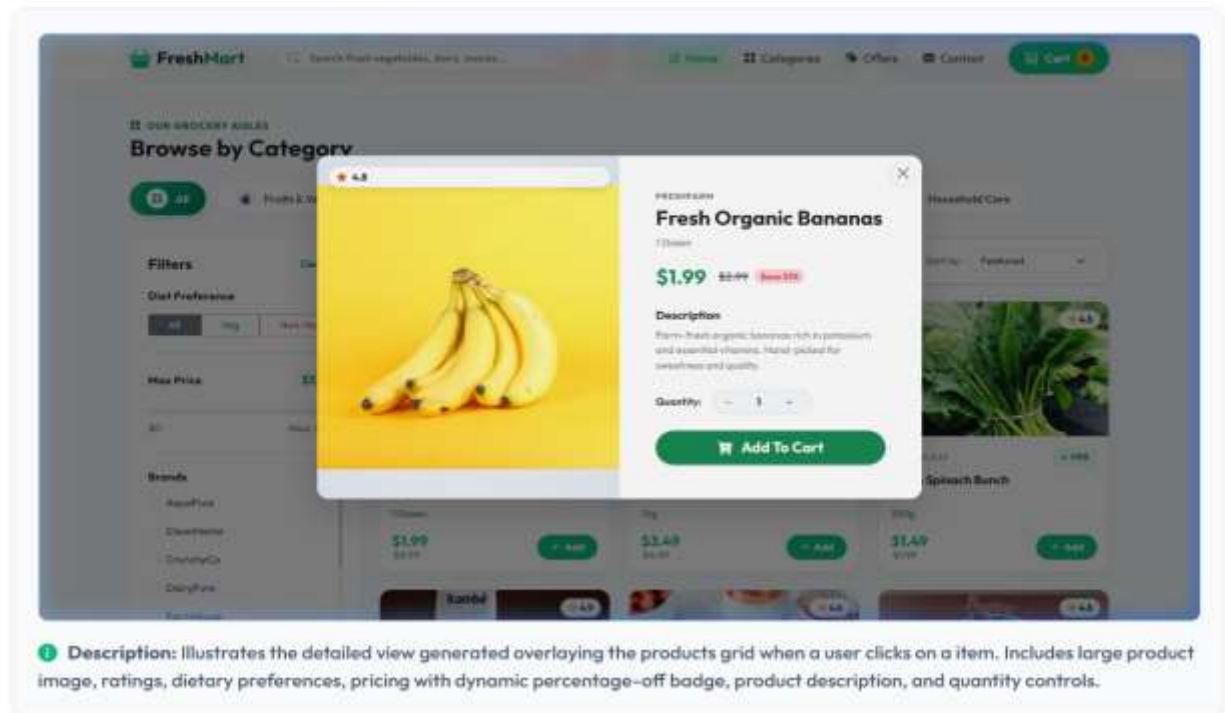
```
// POST /api/orders  
app.post('/api/orders', (req, res) => {  
  const { items, subtotal, deliveryFee, discount, total } = req.body;  
  const trackingCode = `FM-${Math.floor(100000 + Math.random() * 900000)}\`;  
  
  db.run(  
    `INSERT INTO orders (trackingCode, items, subtotal, deliveryFee, discount, total) VALUES  
(?, ?, ?, ?, ?, ?)` ,  
    [trackingCode, JSON.stringify(items), subtotal, deliveryFee, discount, total],  
    function (err) {  
      if (err) return res.status(500).json({ error: err.message });  
      res.status(201).json({ orderId: this.lastID, trackingCode, total });  
    }  
  );  
});
```


Screen-Shots:

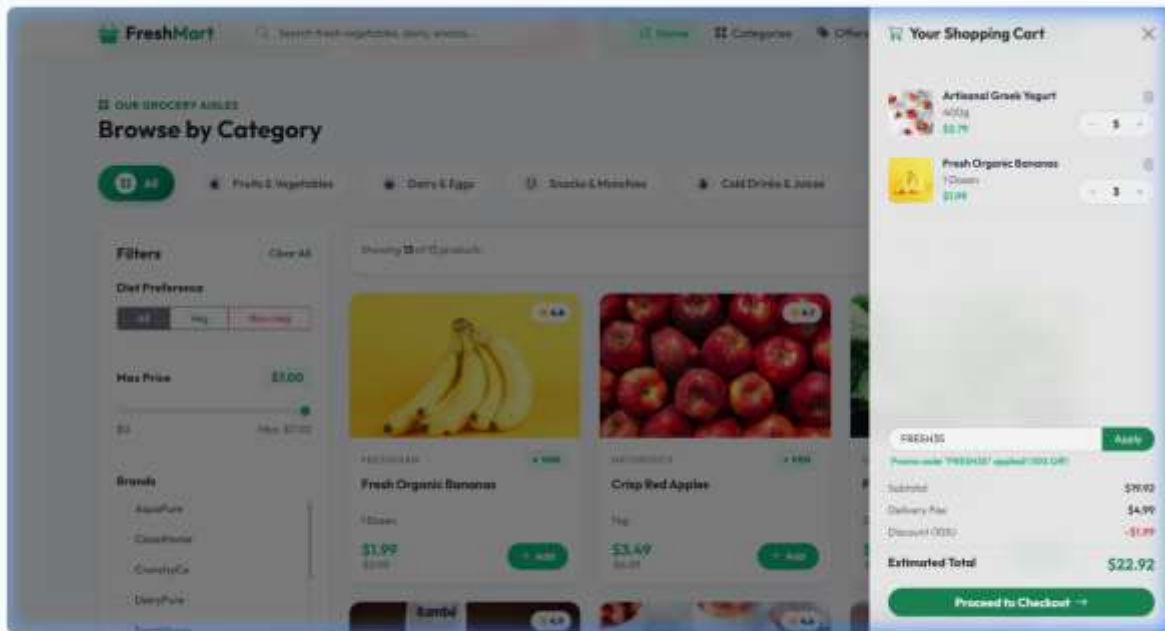
1. Homepage Hero & Promotional Section



2. Product Quick View Modal

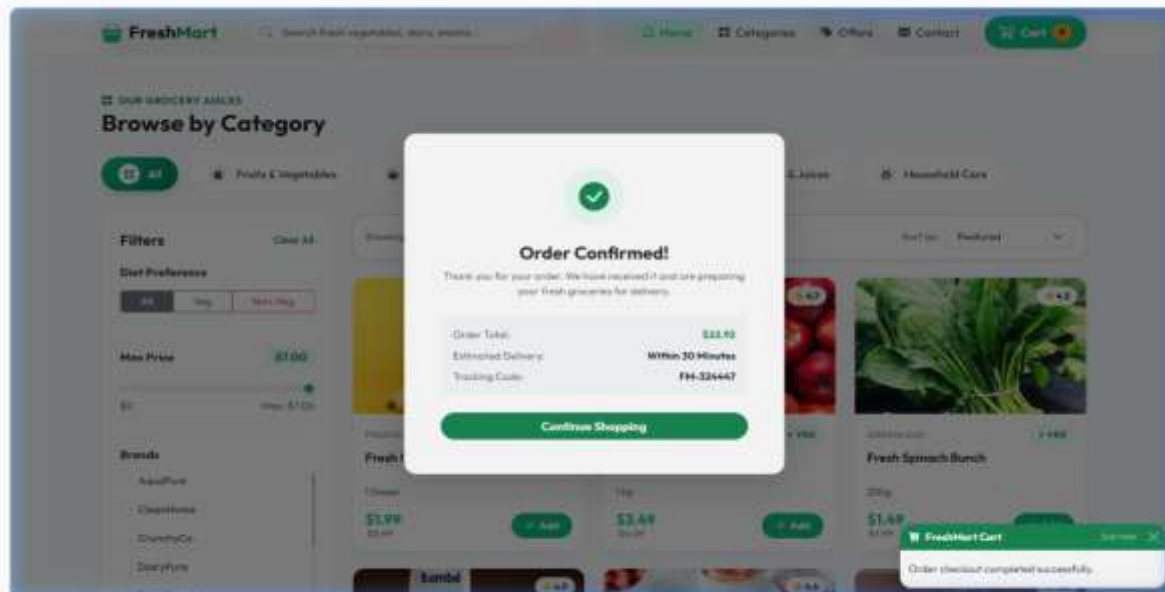


3. Sliding Offcanvas Shopping Cart



Description: Displays the offcanvas drawer containing the product order summary. Item quantities can be adjusted inside the drawer, shipping fee parameters are dynamically updated (free over \$35), and promotional codes (such as code FRESH35) can be verified and applied.

4. Checkout Transaction Order Success



Description: Visualizes the final order success modal presented after posting order items to the SQLite3 database endpoint. It provides the final order total, estimated delivery, and generates a unique order tracking code from the server.

Learning Outcome:

- Built responsive UIs using CSS, Bootstrap, and custom styling.
- Managed application state, cart logic, and local storage.
- Designed SQLite3 databases and wrote SQL queries.
- Developed REST APIs using Node.js and Express.js with CORS support.